

Pipeline CI/CD — CanBankX

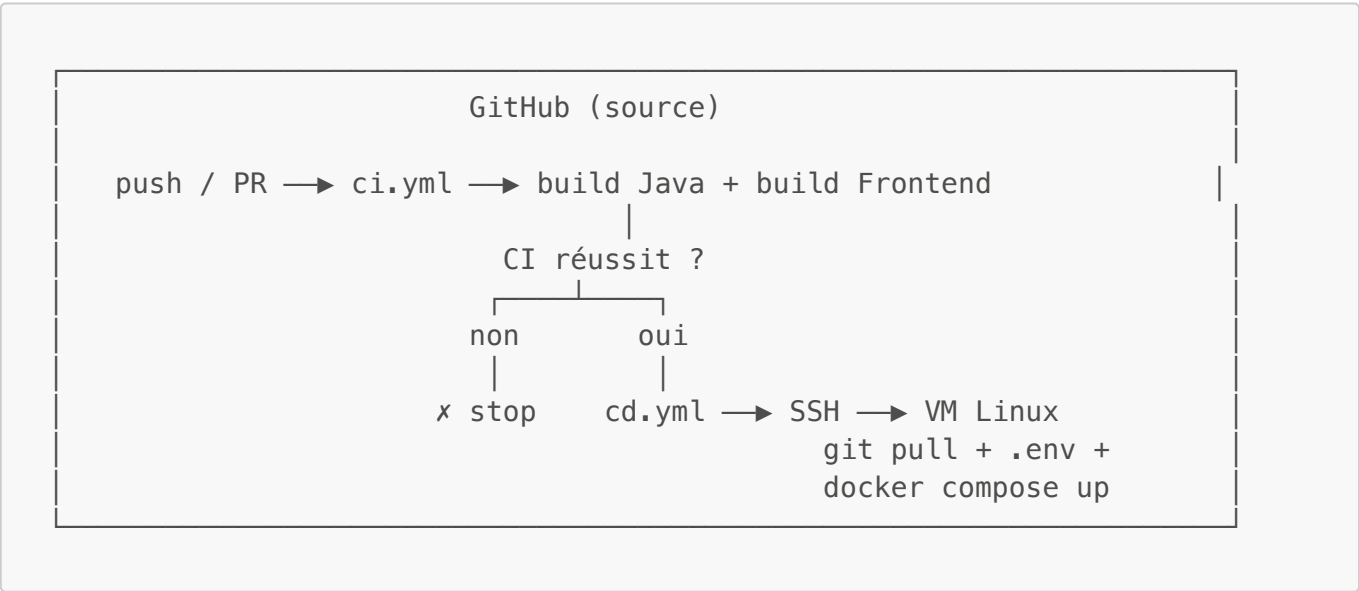
Auteur : Pascal Bourgoin **Date :** Mars 2026 **Compléments :** [arc42.md](#) §8 · [ADR-006-cicd-github-actions](#)

Table des matières

- 1. [Vue d'ensemble](#)
- 2. [Workflow CI — Intégration continue](#)
- 3. [Workflow CD — Déploiement continu](#)
- 4. [Gestion des secrets](#)
- 5. [Script de déploiement \(deploy.sh\)](#)
- 6. [Healthchecks post-déploiement](#)
- 7. [Stratégie de rollback](#)
- 8. [Tests manuels post-déploiement](#)
- 9. [Limites et évolutions possibles](#)

1. Vue d'ensemble

Le projet CanBankX utilise **GitHub Actions** pour automatiser la validation du code (CI) et le déploiement sur une VM Linux via Docker Compose (CD). Les deux pipelines sont séparés afin de garantir qu'aucun déploiement n'a lieu si le build est cassé.



Pipeline	Fichier	Déclencheur	Runner
CI	.github/workflows/ci.yml	push <code>main/develop</code> , PR → <code>main</code>	<code>ubuntu-latest</code>
CD	.github/workflows/cd.yml	CI réussit sur <code>main</code> , <code>workflow_dispatch</code>	<code>ubuntu-latest</code>

2. Workflow CI — Intégration continue

2.1 Déclencheurs

```
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
```

2.2 Job `build-java`

Compile et package tous les modules Maven du projet sans exécuter les tests d'intégration (qui nécessitent MySQL et Redis actifs).

```
jobs:
  build-java:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
      - run: ./mvnw compile -B -q
      - run: ./mvnw package -DskipTests -B -q
```

Les artefacts produits :

- `identity-service/target/*.jar`
- `account-service/target/*.jar`
- `payment-service/target/*.jar`
- `target/LOG430-projet-*.jar`

Nota : `docker build` n'est pas exécuté en CI. Chaque Dockerfile relance Maven en interne ; le faire deux fois serait redondant et doublerait la durée du pipeline.

2.3 Job `build-frontend`

S'exécute **en parallèle** de `build-java` pour réduire la durée totale du CI.

```
build-frontend:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
```

```
node-version: '20'
- working-directory: frontend
  run: npm install
- working-directory: frontend
  run: npm run build
```

2.4 Résumé du CI

Étape	Outil	Durée estimée
Checkout	actions/checkout@v4	~5 s
Compilation Java (tous modules)	./mvnw compile	~45 s
Package Java (sans tests)	./mvnw package -DskipTests	~30 s
Install dépendances frontend	npm install	~20 s
Build frontend	npm run build	~10 s
Total (parallèle)	—	~90 s

3. Workflow CD — Déploiement continu

3.1 Déclencheurs

```
on:
  workflow_run:
    workflows: ["CI"]
    types: [completed]
    branches: [main]
  workflow_dispatch:
```

Le `workflow_dispatch` permet un déclenchement manuel depuis l'interface GitHub sans nécessiter d'accès SSH direct à la VM.

3.2 Condition de déploiement

Le CD **ne s'exécute que si** le CI a réussi :

```
jobs:
  deploy:
    if: ${ github.event.workflow_run.conclusion == 'success' ||
    github.event_name == 'workflow_dispatch' }}
```

3.3 Connexion SSH et script de déploiement

La connexion est établie via l'action `appleboy/ssh-action`. Les secrets sont transmis en clair dans les variables d'environnement SSH (jamais via heredoc pour éviter les problèmes d'échappement).

```
- uses: appleboy/ssh-action@v1
  with:
    host: ${ secrets.VM_HOST }
    username: ${ secrets.VM_USER }
    key: ${ secrets.VM_SSH_KEY }
    envs: DB_ROOT_PASSWORD,DB_USER,DB_PASSWORD
    script: |
      cd ~/log430-projet
      git pull origin main

      printf "DB_ROOT_PASSWORD=%s\nDB_USER=%s\nDB_PASSWORD=%s\n" \
        "$DB_ROOT_PASSWORD" "$DB_USER" "$DB_PASSWORD" > .env

      docker network inspect log430_projet-network >/dev/null 2>&1 \
        || docker network create log430_projet-network

      docker compose down
      docker compose up -d --build

      sleep 30
      curl -sf http://localhost:8081/actuator/health | grep
      "status":"UP"
      curl -sf http://localhost:8082/actuator/health | grep
      "status":"UP"
      curl -sf http://localhost:8083/actuator/health | grep
      "status":"UP"
```

3.4 Séquence de déploiement

- | | |
|--------------------------------------|--|
| 1. git pull origin main | - récupère le dernier commit validé par CI |
| 2. printf > .env
versionner | - écrit les credentials sans les versionner |
| 3. docker network create (si absent) | |
| 4. docker compose down | - arrêt propre des conteneurs (~5 s) |
| 5. docker compose up -d --build | - rebuild des images et redémarrage (~2-4 min) |
| 6. sleep 30 | - attente de l'initialisation Spring |
| 7. curl healthchecks | - vérifie que les 3 services répondent UP |

4. Gestion des secrets

Aucun secret n'est versionné dans le dépôt. Tous sont stockés dans **GitHub Secrets** (Settings → Secrets and variables → Actions).

Secret	Usage
VM_HOST	Adresse IP ou nom DNS de la VM de déploiement
VM_USER	Utilisateur SSH sur la VM
VM_SSH_KEY	Clé privée SSH (RSA ou Ed25519)
DB_ROOT_PASSWORD	Mot de passe root MySQL
DB_USER	Utilisateur applicatif MySQL
DB_PASSWORD	Mot de passe de l'utilisateur applicatif MySQL

Renouvellement : Si la VM est recréée, il faut régénérer la paire de clés SSH, ajouter la clé publique dans `~/.ssh/authorized_keys` sur la VM, et mettre à jour le secret `VM_SSH_KEY` dans GitHub.

5. Script de déploiement (deploy.sh)

Le fichier `deploy.sh` à la racine du projet est un **shim informatif** : le déploiement automatique est entièrement délégué au CD pipeline.

```
#!/usr/bin/env bash
# Le déploiement est géré par GitHub Actions (.github/workflows/cd.yml).
echo "Le déploiement automatique passe par GitHub Actions (cd.yml)."
```

```
echo "Pour un déploiement manuel, utiliser : docker compose up -d --build"
```

Pour un déploiement **entièrement manuel** (hors pipeline), se connecter à la VM et exécuter :

```
cd ~/log430-projet
git pull origin main
# Créer/mettre à jour le .env manuellement
docker compose up -d --build
```

6. Healthchecks post-déploiement

Chaque service Spring Boot expose `/actuator/health` via Spring Boot Actuator. Docker Compose effectue ses propres healthchecks internes ; le pipeline CD ajoute une vérification externe après le démarrage.

Service	Port	Endpoint healthcheck
identity-service	8081	<code>http://localhost:8081/actuator/health</code>
account-service	8082	<code>http://localhost:8082/actuator/health</code>
payment-service	8083	<code>http://localhost:8083/actuator/health</code>

Réponse attendue :

```
{ "status": "UP" }
```

Si l'un des `curl` échoue, le job CD se termine en erreur, ce qui alerte l'équipe via la notification GitHub Actions (e-mail / Slack selon la configuration du dépôt).

7. Stratégie de rollback

Le rollback s'effectue via Git, sans outillage supplémentaire :

```
# 1. Identifier le commit stable précédent
git log --oneline -10

# 2. Créer un commit de revert
git revert <commit-sha>

# 3. Push sur main → déclenche CI puis CD automatiquement
git push origin main
```

Le pipeline CI revalide le code issu du revert, puis CD redéploie. La fenêtre de rollback est identique à celle d'un déploiement normal (~3-5 min).

Alternative rapide : En cas d'urgence, se connecter directement à la VM, faire `git checkout <tag-stable>` puis `docker compose up -d --build`. Cette procédure bypass le pipeline.

8. Tests manuels post-déploiement

Les tests de charge (k6) nécessitent MySQL, Redis, MailHog et KrakenD actifs simultanément. Ils ne peuvent pas être exécutés dans les runners GitHub Actions (sans services Docker composés). Ils sont lancés **manuellement sur la VM** après chaque déploiement réussi.

Test	Fichier	Commande	Durée
Smoke	k6/smoke-test.js	k6 run k6/smoke-test.js	~30 s
Load (50 VUs)	k6/load-test.js	k6 run k6/load-test.js	~3 min
Stress (200 VUs)	k6/stress-test.js	k6 run k6/stress-test.js	~5 min

Les résultats sont analysés dans `observabilite-charge.md`.

9. Limites et évolutions possibles

Limite actuelle	Évolution possible
-----------------	--------------------

Limite actuelle	Évolution possible
Fenêtre d'indisponibilité ~15-30 s (down + up)	Déploiement blue/green avec Nginx upstream switch
Tests k6 lancés manuellement	Job CD avec services : Docker Compose ou environnement dédié
Un seul environnement (production)	Ajouter un environnement staging déclenché sur develop
Pas de cache Maven en CI	actions/cache sur ~/ .m2 pour réduire le temps de build (~30 s économisés)
Pas de scan de vulnérabilités	Intégrer trivy ou snyk sur les images Docker en CI
Notifications limitées aux e-mails GitHub	Intégrer Slack webhook sur échec de pipeline